

Esercizio Lambda Expressions

Il Contesto

Siamo nel 2250. L'Impero Galattico ha bisogno di reclutare nuovi specialisti, ma il database è pieno di profili inutili. Il vostro compito è scrivere un sistema di filtraggio ultraveloce utilizzando le **Lambda Expressions** e le **Functional Interfaces** di Java.

Requisiti Preliminari

Si consideri la classe base Candidato:

```
public class Candidato {  
    private String nome;  
    private int livelloForza; // 1-100  
    private double costoIngaggio;  
    private String specializzazione; // "Pilota", "Meccanico",  
    "Diplomatico", "Soldato"  
    private boolean possiedeNave;  
  
    // Costruttore, Getters e ToString...  
}
```

N.B. Per l'attributo "specializzazione" è possibile utilizzare un enumerativo al posto del tipo String.

Crea una lista di 1000 candidati generati casualmente. Crea un metodo per generare casualmente un candidato.

Task 1: Il filtro manuale (Predicate)

Invece di creare mille metodi filterByX, implementa un metodo generico che accetti un'interfaccia funzionale.

1. Utilizzare `java.util.function.Predicate<Candidato>`.
2. Scrivere un metodo `filtraCandidati(List<Candidato> lista, Predicate<Candidato> criterio)`.
3. **Esercizio:** Trovare tutti i "Piloti" con `livelloForza > 70` utilizzando il metodo realizzato in precedenza.

Task 2: La tassa Imperiale (Function)

L'Impero applica una tassa sul costo di ingaggio in base alla specializzazione.

1. Creare un metodo che calcoli il costo totale di ingaggio:
`costoIngaggio(List<Candidato> lista, java.util.function.Function<Candidato, Double> f)`.

2. Utilizzare il metodo precedente creando una Lambda che calcoli il costo finale tenendo conto del costo di ingaggio di ogni candidato nella lista. Se il candidato è un "Soldato", il costo aumenta del 10%, altrimenti resta invariato.

Task 3: Il Grido di battaglia (Consumer)

Ogni candidato deve presentarsi.

1. Crea un metodo gridoDiBattaglia(List<Candidato> lista, java.util.function.Consumer<Candidato> c).
2. Utilizzare il metodo precedente creando una Lambda che stampi in console: "[NOME]: Sono pronto a servire l'Impero!" in maiuscolo se la forza è > 80, o in minuscolo se è inferiore.

Task 4: Ordinamento dinamico (Comparator)

L'Impero è meritocratico (o quasi).

1. Ordinare la lista dei candidati con list.sort(). Il metodo sort accetta un'interfaccia funzionale Comparator.
2. Passare una Lambda che implementi Comparator<Candidato>.
3. Ordinare per livelloForza decrescente e, a parità di forza, per costoIngaggio crescente.

Task 5: La "Great Selection"

Utilizzando gli stream realizzare una pipeline sulla lista di candidati che:

- Filtra solo chi ha una nave.
- Mappa i nomi in una lista di stringhe.
- Stampa i primi 3 profili più economici.

Task 6: La pipeline di selezione

Dobbiamo generare un report dei soli "Top Gun".

1. Partire dalla lista dei candidati.
2. Creare una pipeline che:
 - Filtra i candidati con livelloForza > 80.
 - Estrae solo il nome (trasformando lo stream in Stream<String>).
 - Raccoglie i risultati in una List<String> usando .collect(Collectors.toList()).

Task 7: Il bilancio dell'Impero (Reduction) – 20 min

L'Imperatore è avaro. Dobbiamo calcolare l'investimento totale e trovare l'eccellenza.

1. **Somma totale:** calcolare il costo totale d'ingaggio di tutti i candidati usando `.mapToDouble()` e `.sum()`.
2. **Il massimo:** trovare il candidato con il livelloForza più alto usando `.reduce()` o `.max(Comparator)`.
3. **Media scientifica:** calcolare la forza media del gruppo.

Task 8: Intelligence e logistica (`Collectors.groupingBy`)

Dobbiamo raggruppare i candidati per capire dove sono le risorse.

1. **Raggruppamento semplice:** utilizzare `Collectors.groupingBy(Candidato::getSpecializzazione)` per ottenere una `Map<String, List<Candidato>>` e raggruppare i candidati in base alla specializzazione.
2. **Conteggio per ruolo:** modificare il collector per ottenere una mappa che associa a ogni specializzazione il numero di candidati presenti: `Map<String, Long>`.
3. **Analisi avanzata:** creare una mappa che associa ogni specializzazione al candidato più economico della relativa specializzazione.